

ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»

Факультет компьютерных наук
Образовательная программа «Компьютерные науки и анализ данных»

ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА

Программный проект на тему:
Веб-приложение для группового общения

Выполнил студент:

группы #БКНАД221, 4 курса

Деев Семен Алексеевич

Принял руководитель ВКР:

Промыслов Валентин Валерьевич

Доцент

Факультет компьютерных наук НИУ ВШЭ

Соруководитель:

Овчинников Сергей Андреевич

Ведущий инженер

ООО "ВымпелКом-Информационные технологии"

Содержание

Аннотация

В выпускной квалификационной работе рассматривается проектирование и реализация веб-приложения «Chatterly», предназначенного для группового общения пользователей в рамках онлайн-сообществ. Актуальность работы обусловлена потребностью в доступных браузерных коммуникационных сервисах, которые объединяют постоянные сообщества, тематические каналы, историю переписки и групповые звонки, но не требуют установки отдельного клиента и не обладают избыточной сложностью интерфейса.

Целью работы является разработка прототипа веб-приложения для организации пользовательских сообществ, личной переписки, обмена текстовыми сообщениями в реальном времени и проведения аудио- и видеозвонков. В ходе работы проведён анализ существующих решений, сформированы функциональные и нефункциональные требования, спроектирована архитектура системы и реализованы основные компоненты приложения.

Серверная часть приложения реализована на языке Go, клиентская часть – с использованием JavaScript и SolidJS. Для хранения данных применяется PostgreSQL, для сессий, кэширования и публикации событий – Redis, для работы с пользовательскими изображениями – S3-совместимое хранилище. Обмен событиями в реальном времени реализован на основе WebSocket, а аудио- и видеосвязь – на основе WebRTC.

Результатом работы является прототип коммуникационного веб-приложения, включающий управление пользователями, серверы и топики, личные сообщения, историю переписки, доставку сообщений в реальном времени и базовую поддержку групповых звонков. В работе также рассмотрены ограничения текущей реализации и направления дальнейшего развития системы.

Ключевые слова

Веб-приложение, групповая коммуникация, обмен сообщениями в реальном времени, видеосвязь, Go, JavaScript, PostgreSQL, Redis, WebSocket, WebRTC.

1 Введение

Современные цифровые сообщества в значительной степени опираются на программные средства сетевой коммуникации. Такие средства используются в образовательных группах, профессиональных командах, игровых сообществах, клубах по интересам и иных объединениях, где пользователям необходимо не только обмениваться отдельными сообщениями, но и поддерживать устойчивую структуру общения. Для подобных сценариев важны постоянные пространства взаимодействия, тематическое разделение обсуждений, история переписки, оперативная доставка новых сообщений, а также возможность голосовой и видеосвязи.

Развитие веб-технологий привело к тому, что значительная часть коммуникационных функций может быть реализована непосредственно в браузере. Пользовательский сценарий в этом случае становится проще: для доступа к системе достаточно открыть веб-страницу, пройти регистрацию или авторизацию и перейти к нужному сообществу или диалогу. Такой подход особенно важен для пользователей, которым требуется быстрое подключение к общению без установки отдельного клиента и сложной настройки окружения.

Одним из наиболее близких по назначению решений является Discord, предоставляющий модель серверов, каналов, личных сообщений и голосовых комнат [discord]. Однако использование Discord как универсальной основы для целевой аудитории ограничено двумя обстоятельствами. Во-первых, интерфейс и система настроек платформы могут быть избыточными для пользователей, которым необходим базовый набор функций группового общения. Во-вторых, доступ к Discord на территории Российской Федерации ограничен [discord_russia]. Следовательно, несмотря на функциональную близость к целевой модели, Discord не устраняет полностью рассматриваемую практическую проблему.

Другие распространённые решения также закрывают только отдельные части задачи. Zoom и Jitsi Meet удобны для видеоконференций и разовых встреч, но не ориентированы на модель постоянных сообществ с тематическими каналами и долговременной историей обсуждений [zoom, jitsi]. TeamSpeak хорошо решает задачу голосового общения, однако в меньшей степени соответствует современным требованиям к браузерному интерфейсу, видеосвязи и текстовому взаимодействию [teamspeak]. Matrix и Element предоставляют развитую модель независимой коммуникационной инфраструктуры и контроля данных, но могут быть сложнее для массового пользователя с точки зрения настройки и освоения [matrix, element]. Таким образом, существующие аналоги либо недоступны, либо сложны, либо ориентированы на более узкий коммуникационный сценарий.

Актуальность настоящей работы определяется необходимостью разработки доступного веб-приложения, которое объединяет базовые возможности современных коммуникационных платформ: создание сообществ, разделение общения по тематическим каналам, личные сообщения, хранение истории, обмен событиями в реальном времени и групповые аудио-/видеозвонки. При этом особое значение имеет простота пользовательского интерфейса, поскольку целевое приложение должно быть пригодно не только для технически подготовленных пользователей, но и для обычных участников онлайн-сообществ.

Объектом работы является веб-приложение для группового общения пользователей. Предметом работы являются архитектурные и программные решения, необходимые для реализации такого приложения: модель данных, серверная бизнес-логика, клиентский интерфейс, механизм доставки событий в реальном времени и механизм установления мультимедийных соединений между участниками звонка.

Целью выпускной квалификационной работы является проектирование и реализация прототипа веб-приложения «Chatterry», предназначенного для организации группового общения пользователей в рамках сообществ, структурированных по тематическим топикам.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1 провести анализ существующих коммуникационных приложений и определить их ограничения относительно целевого сценария;
- 2 сформировать функциональные и нефункциональные требования к разрабатываемой системе;
- 3 спроектировать архитектуру серверной части, клиентской части, базы данных, WebSocket-контура и подсистемы аудио-/видеозвонков;
- 4 реализовать серверную часть приложения на языке Go с REST API, WebSocket-интерфейсом, доступом к PostgreSQL, Redis и S3-совместимому хранилищу;
- 5 реализовать клиентскую часть приложения на JavaScript и SolidJS, включая страницы авторизации, личных сообщений, серверов, текстовых и голосовых топиков;
- 6 реализовать обмен текстовыми сообщениями в реальном времени и сохранение истории сообщений;
- 7 реализовать базовую поддержку групповых аудио- и видеозвонков с использованием WebRTC;

8 описать подход к тестированию и оценить соответствие полученного прототипа сформированным требованиям.

Практическая значимость работы состоит в создании прототипа независимого веб-приложения, которое может использоваться как основа для дальнейшего развития коммуникационной платформы. В отличие от решений, ориентированных только на видеовстречи или только на голосовое общение, разрабатываемая система объединяет несколько связанных пользовательских сценариев: постоянные сообщества, тематические топики, личную переписку, историю сообщений и звонки. В отличие от функционально насыщенных коммерческих платформ, Chatterly рассматривается как более простой по пользовательской модели прототип, в котором основное внимание уделяется базовым сценариям группового взаимодействия.

Техническая значимость работы связана с интеграцией нескольких механизмов, характерных для современных приложений реального времени. Серверная часть реализована на языке Go и построена по слоистой архитектуре: выделены доменные модели, сервисы бизнес-логики, адаптеры к внешним хранилищам и HTTP/WebSocket API. Для хранения постоянных данных используется PostgreSQL, для кэширования сессий и публикации событий – Redis, для пользовательских изображений – S3-совместимое объектное хранилище. Доставка событий между пользователями реализована на основе WebSocket, а передача аудио- и видеопотоков – на основе WebRTC.

В результате работы разработан прототип приложения, включающий регистрацию и аутентификацию пользователей, управление профилем, создание серверов и топиков, личные сообщения, текстовые сообщения в топиках, постраничную загрузку истории, уведомления о новых сообщениях и пользовательский интерфейс голосовых топиков. Также реализована серверная обработка событий звонков, включая подключение участников, передачу сигнальных сообщений и обработку медиапотоков.

2 Анализ существующих приложений

Для обоснования разработки Chatterly недостаточно сравнить аналоги только по принципу наличия или отсутствия отдельных функций. Коммуникационные приложения отличаются не только набором возможностей, но и основной моделью взаимодействия: одни строятся вокруг постоянных сообществ, другие – вокруг разовой встречи, третьи – вокруг федеративной инфраструктуры или голосового канала. Поэтому в данном разделе рассматриваются как пользовательские функции, так и архитектурные следствия выбранной модели.

Сравнение проводится по следующим критериям:

- 1 основная пользовательская модель: постоянное сообщество, личная переписка, встреча, комната или голосовой канал;
- 2 структура коммуникации: наличие серверов, тематических текстовых и голосовых пространств, ролей и участников;
- 3 поддержка сообщений в реальном времени, истории переписки и уведомлений;
- 4 поддержка групповой аудио- и видеосвязи, демонстрации экрана и браузерного подключения;
- 5 модель развёртывания и контроля данных: централизованный сервис, самостоятельное развёртывание или федерация;
- 6 сложность освоения и доступность для целевой аудитории.

Такой набор критериев позволяет связать анализ аналогов с последующими требованиями к Chatterry: если аналог закрывает отдельный сценарий, но не подходит как целостная основа, из этого формулируется конкретное проектное решение для разрабатываемой системы.

2.1 Discord

Discord является наиболее близким функциональным аналогом Chatterry. Его основная пользовательская модель строится вокруг сервера как постоянного цифрового пространства, в котором участники объединяются по интересам, учёбе, работе или другой общей цели. Внутри сервера создаются каналы разных типов: текстовые каналы для обсуждений, голосовые каналы для общения и видеосвязи, а также дополнительные формы организации обсуждений, например категории и треды. Для крупных сообществ Discord также предоставляет роли, права доступа, модерацию, приглашения, onboarding и другие средства управления участниками [discord].

Сильная сторона Discord состоит в том, что структура общения совпадает с естественной структурой многих онлайн-сообществ: есть общее пространство, внутри него – тематические каналы, а пользователи могут переходить между текстовым и голосовым взаимодействием. Для проектируемого приложения эта модель важна как предметный ориентир: Chatterry также должен иметь сущности пользователя, сервера, участника, текстового топика, голосового топика и сообщений.

Ограничения Discord связаны не с отсутствием функций, а с применимостью продукта к целевому сценарию. Во-первых, развитая система ролей, настроек, интеграций и дополнительных инструментов повышает сложность интерфейса для пользователя, которому требуется только базовое групповое общение. Во-вторых, сервис является централизованной внешней платформой, поэтому пользователь и организация не контролируют развёртывание и хранение данных. В-третьих, доступ к Discord в Российской Федерации был ограничен Роскомнадзором [discord_russia]. Следовательно, для Chatterry целесообразно использовать продуктовую идею серверов и каналов, но реализовать её как более простой независимый MVP с минимальным набором ролей и сценариев.

2.2 Zoom

Zoom ориентирован прежде всего на видеоконференции, онлайн-встречи и вебинары. Важными возможностями платформы являются подключение участников к встрече, аудио- и видеосвязь, демонстрация экрана, управление участниками и инструменты для корпоративной коммуникации [zoom]. В составе Zoom Workplace также существует Team Chat: он поддерживает личные сообщения, групповые чаты и каналы, которые могут использоваться как долгосрочные пространства для проектов или команд [zoom_chat].

Таким образом, Zoom нельзя корректно описывать как систему, в которой полностью отсутствует текстовое взаимодействие. Однако его ключевая модель всё равно отличается от целевого сценария Chatterry. Zoom исторически и продуктово строится вокруг встречи или корпоративной рабочей среды, а не вокруг пользовательского сообщества, где текстовые топики, голосовые комнаты, история сообщений и членство в сервере являются равноправными частями одной предметной модели. Каналы Zoom Team Chat полезны для рабочих групп внутри аккаунта организации, но они не образуют модель пользовательских серверов, аналогичную Discord.

Для Chatterry из анализа Zoom следует два вывода. Первый вывод состоит в том, что видеосвязь должна иметь низкий барьер входа и поддерживать привычные действия пользователя: включение микрофона, камеры и демонстрации экрана. Второй вывод состоит в том, что звонок не должен становиться единственной центральной сущностью приложения. В Chatterry аудио- и видеосвязь должны быть встроены в постоянное сообщество и запускаться в контексте голосового топики, рядом с текстовыми топиками и историей сообщений. Для российской аудитории также сохраняется общий риск зависимости от внешнего коммерческого сервиса [zoom_russia].

2.3 Jitsi Meet

Jitsi Meet представляет собой открытое решение для аудио- и видеоконференций. Сервис позволяет создавать встречу по ссылке, подключать участников без обязательной регистрации, демонстрировать рабочий стол, использовать встроенный чат и при необходимости разворачивать собственный экземпляр системы [jitsi]. По сравнению с коммерческими видеосервисами Jitsi особенно важен как пример браузерной доступности и самостоятельного развёртывания.

Архитектурно Jitsi Meet решает задачу медиа-сеанса, а не задачу длительного сообщества. Центральной сущностью является встреча с URL, к которой подключаются участники. Такая модель хорошо подходит для быстрого звонка, но в ней нет полноценной предметной области, необходимой Chatterry: серверов пользователя, набора постоянных текстовых и голосовых топиков, истории сообщений по каждому топикю, профилей участников и личных диалогов. Встроенный чат Jitsi полезен внутри встречи, но не заменяет долгосрочную переписку сообщества.

Из Jitsi Meet для Chatterry следует сохранить идею работы через браузер и минимального барьера подключения к звонку. При этом звонок должен быть не самостоятельной одноразовой комнатой, а частью постоянного пространства общения. Поэтому в Chatterry медиасоединение реализуется через WebRTC, а сигнальные события и состояние участников привязываются к голосовому топикю и доставляются через общий real-time контур приложения.

2.4 Matrix / Element

Matrix отличается от остальных рассмотренных решений тем, что является не отдельным приложением, а открытым протоколом для децентрализованной коммуникации. Спецификация Matrix описывает открытые API для обмена сообщениями, VoIP-сигналикации и синхронизации данных в федерации серверов. В этой модели пользователь работает через клиент, клиент обращается к домашнему серверу, а история и состояние комнат синхронизируются между серверами-участниками [matrix]. Element является одним из наиболее известных клиентов и платформенных решений на основе Matrix; он ориентирован на защищённую коммуникацию, контроль данных, сквозное шифрование, развёртывание в облаке или on-premise [element, element_privacy].

Сильная сторона Matrix / Element состоит в независимости от единого поставщика и в формализованной архитектуре обмена событиями. Для Chatterry особенно важны идеи ком-

нат, событий, профилей, истории сообщений и разделения клиент-серверного API от межсерверного взаимодействия. В то же время федеративная архитектура существенно повышает сложность реализации: необходимо учитывать синхронизацию состояния между серверами, идентичность пользователей в разных доменах, устройства, ключи шифрования, согласование истории и обработку конфликтов.

Для прототипа Chatterly такая сложность избыточна. Цель работы состоит не в реализации открытого федеративного протокола, а в создании независимого веб-приложения для базовых сценариев группового общения. Поэтому из Matrix / Element следует заимствовать не федерацию как обязательное требование, а архитектурный принцип событийной коммуникации и контроля данных. В MVP Chatterly это выражается в централизованной серверной части, собственной базе данных, WebSocket-событиях для доставки изменений и возможности дальнейшего развития системы без зависимости от конкретной зарубежной платформы.

2.5 TeamSpeak

TeamSpeak ориентирован прежде всего на голосовое общение. Базовый пользовательский сценарий состоит в подключении к серверу и переходе в канал, где участники могут разговаривать друг с другом; голосовые данные передаются через сервер [teamspeak]. Такая модель исторически хорошо подходит для игровых и технических сообществ, которым важны постоянные голосовые комнаты, качество аудио и управление доступом.

Основное ограничение TeamSpeak – узкая направленность. В Chatterly текстовые сообщения, личные диалоги, история и звонки должны быть частью единого браузерного приложения. TeamSpeak в первую очередь решает задачу голосовых каналов. Даже при наличии дополнительных средств общения его основная предметная модель не делает текстовые топики, историю сообщений и видеосвязь равноправными объектами системы.

Для Chatterly TeamSpeak подтверждает важность постоянных голосовых комнат: пользователю удобно не создавать новую встречу каждый раз, а заходить в уже существующий голосовой топик внутри сообщества. Однако эта идея должна быть реализована в современной веб-модели: пользователь открывает приложение в браузере, выбирает сервер и голосовой топик, а передача медиа выполняется через WebRTC при сохранении связи с остальными компонентами системы.

Сводная таблица

Решение	Основная модель	Полезные свойства	Ограничения для целевого сценария	Вывод для Chatterly
Discord	постоянные серверы с каналами, ролями и участниками	серверы, текстовые и голосовые каналы, видеосвязь, права доступа, модерация	зависимость от внешней централизованной платформы, высокая насыщенность интерфейса, ограниченная доступность в РФ	использовать модель серверов и топиков, но упростить MVP и сохранить независимость реализации
Zoom	встреча и корпоративная рабочая среда; Team Chat поддерживает каналы	устойчивый сценарий видеовстреч, демонстрация экрана, быстрый вход в звонок, рабочие чат-каналы	каналы не образуют модель пользовательских серверов с текстовыми и голосовыми топиками; звонок остаётся центральным сценарием	встроить аудио- и видеосвязь в постоянное сообщество, а не строить приложение только вокруг встречи
Jitsi Meet	браузерная встреча по ссылке	открытый код, простое подключение, самостоятельное развёртывание, встроенный чат во время звонка	нет полноценной модели серверов, профилей, топиков и долговременной истории сообщений	использовать браузерную доступность звонков, но связать WebRTC с постоянными топиками и историей
Matrix / Element	федерация домашних серверов, комнаты и события	открытый протокол, контроль данных, сквозное шифрование, on-premise и cloud-развёртывание	высокая архитектурная и пользовательская сложность для MVP: федерация, синхронизация, устройства и ключи	сохранить событийную модель и контроль данных, но реализовать централизованный прототип
TeamSpeak	серверы и голосовые каналы	постоянные голосовые комнаты, управление доступом, ориентация на низкую задержку аудио	голосовое общение является главным сценарием; текстовая история и видеосвязь не являются равноправной основой продукта	реализовать голосовые топиксы внутри веб-приложения вместе с текстовыми сообщениями и видеосвязью

Таблица 2.1: Сравнение существующих решений по модели взаимодействия и выводам для проекта

Сравнение в таблице ?? показывает, что ни одно из рассмотренных решений не закрывает целевой сценарий полностью. Из проведённого анализа следует, что Chatterly должен объединить несколько свойств в одной системе: постоянные пользовательские серверы, текстовые и голосовые топиксы, личные сообщения, сохранение истории, доставку событий в реальном времени и браузерные аудио- и видеозвонки. На уровне архитектуры это приводит к выбору централизованного клиент-серверного MVP: PostgreSQL используется для постоянного состояния и истории, WebSocket – для доставки сообщений и событий звонков, WebRTC – для передачи аудио- и видеопотоков, а пользовательский интерфейс ограничивается базо-

выми сценариями без сложной системы ролей и федерации. Эти выводы используются далее при формировании требований к приложению.

3 Формирование требований к приложению

Требования к Chatterly формируются на основе анализа аналогов. Функциональные требования описывают поведение системы и пользовательские возможности. Нефункциональные требования фиксируют ограничения и качественные характеристики. Для удобства дальнейших ссылок требования имеют идентификаторы **F** для функциональных требований и **NF** для нефункциональных требований.

Вывод из анализа	Где проявляется	Требования	Архитектурное следствие
Необходима модель постоянных сообществ, а не только разовых встреч	Discord, Zoom, Jitsi Meet	F-3, F-4, NF-2	выделяются сущности сервера, участника и топика; структура сервера хранится в PostgreSQL
Текстовое общение, звонки и история должны быть частью одной системы	Discord, Zoom, Jitsi Meet, TeamSpeak	F-5, F-6, F-7, F-9	сообщения сохраняются в БД, а звонки привязываются к голосовым топикам
Для пользовательского опыта важна доставка событий в реальном времени	Discord, Matrix / Element, все чаты	F-8, F-10, NF-4	используется единый WebSocket endpoint и Redis pub/sub для доставки событий пользователям
Браузерная аудио- и видеосвязь должна быть встроена в приложение	Discord, Zoom, Jitsi Meet	F-9, F-10, NF-1	медиапоток передается через WebRTC, а сигнальные сообщения обрабатываются сервером
Сложные роли, федерация и избыточные настройки не требуются для MVP	Discord, Matrix / Element	F-3, F-11, NF-2, NF-5	выбирается централизованная архитектура с ролями owner/member и простым пользовательским сценарием
Система должна контролировать данные и доступ к действиям	Matrix / Element, Discord	F-1, F-2, NF-3, NF-6	данные хранятся в PostgreSQL и S3-совместимом хранилище; доступ проверяется на серверной стороне

Таблица 3.1: Прослеживаемость требований от анализа аналогов и выводов преддипломной практики

3.1 Функциональные требования

Функциональные требования ниже описывают не только пользовательскую возможность, но и ожидаемое поведение серверной и клиентской частей. Это необходимо для последующей проверки реализации и для устранения недостатка преддипломной практики, где требования были сформулированы без достаточной детализации сценариев и API.

F-1 Регистрация и аутентификация пользователей. Система должна позволять со-

здать учётную запись по имени пользователя, логину и паролю, выполнить вход, выход и получить сведения о текущем пользователе. Серверная часть должна проверять уникальность логина и имени пользователя, хранить пароль в виде хэша, создавать сессию после успешного входа и передавать клиенту session cookie. На уровне API этому соответствуют операции создания пользователя, входа, выхода и получения текущей сессии.

F-2 Управление пользовательским профилем. Пользователь должен иметь возможность просматривать текущий профиль, изменять имя, логин и пароль, удалять профиль, загружать и удалять аватар. При изменении логина или пароля сервер должен требовать текущий пароль. Пользовательское изображение должно храниться во внешнем S3-совместимом хранилище; при отсутствии аватара система должна генерировать identicon, чтобы пользователь оставался визуально различимым в списках и сообщениях.

F-3 Сообщества и участники. Приложение должно поддерживать создание сервера как постоянного пространства общения, получение списка серверов пользователя, поиск серверов, к которым пользователь ещё не присоединился, вступление в сервер и выход из него. Пользователь, создавший сервер, получает роль владельца; остальные участники получают роль member. Изменение и удаление сервера должны быть доступны только владельцу. В рамках MVP серверы рассматриваются как доступные через поиск пространства, без отдельной модели приватных приглашений.

F-4 Тематические топики. Внутри сервера должны поддерживаться топики двух типов: текстовый топик для сообщений и голосовой топик для звонков. Владелец сервера должен иметь возможность создавать, изменять и удалять топики. Система должна запрещать дублирование топиков с одинаковым именем и типом внутри одного сервера, а также должна проверять, что пользователь имеет доступ к серверу перед входом в топик.

F-5 Личные сообщения. Пользователь должен иметь возможность искать других пользователей, с которыми у него ещё нет личного диалога, создавать личный диалог, просматривать список диалогов и переходить к переписке. В списке диалогов должны отображаться собеседник, последнее сообщение и признак непрочитанности. Система должна запрещать создание диалога пользователя с самим собой и повторное создание уже существующего DM.

- F-6 Отправка и хранение текстовых сообщений.** Пользователь должен иметь возможность отправлять сообщения в личный диалог и текстовый топик. Сервер должен проверять доступ пользователя к соответствующему DM или топику, сохранять сообщение в PostgreSQL, обновлять состояние последнего сообщения и передавать событие участникам через real-time контур. Для отправки используются отдельные REST-операции для DM и сообщений текстового топика.
- F-7 История сообщений и пагинация.** Чат должен поддерживать загрузку первой страницы истории и последующих страниц при прокрутке вверх. Пагинация должна быть стабильной при наличии сообщений с одинаковым временем создания, поэтому cursor должен учитывать идентификатор чата или топика, время создания и идентификатор последнего загруженного сообщения. Это требование обеспечивает постоянный контекст общения, которого недостаточно в сервисах, ориентированных только на разовые встречи.
- F-8 WebSocket-подписки и уведомления.** Клиент должен подключаться к единому WebSocket endpoint `/ws/`. Соединение должно поддерживать события `join`, `leave`, `ping`, `pong`, `message` и `error`, а также каналы типов `dm`, `text_topic` и `voice_topic`. Перед подпиской сервер должен проверять доступ пользователя к каналу. При входящем личном сообщении интерфейс должен обновлять список диалогов, признак непрочитанности и показывать уведомление, если пользователь находится в другом чате.
- F-9 Голосовые и видеозвонки.** Пользователь должен иметь возможность подключаться к голосовому топику сервера и участвовать в групповом звонке. Клиентская часть должна поддерживать передачу микрофона, камеры и демонстрации экрана через WebRTC, отображение локального preview, плитки удалённых участников, выбор устройств и обработку ошибок доступа к media devices.
- F-10 WebRTC signaling и состояние голосового топика.** Для установления WebRTC-соединений сервер и клиент должны обмениваться сигнальными событиями семейства `voice_*`: `offer`, `answer`, `ICE candidates`, состояние участников, вход и выход пользователя из голосового топика. Серверная часть должна хранить состояние активных участников, обрабатывать отключение пользователя и рассылать изменения остальным участникам звонка.
- F-11 Пользовательский интерфейс основных сценариев.** Клиентская часть должна предоставлять страницы регистрации и входа, список личных диалогов, поиск пользо-

вателей, список серверов, поиск и создание серверов, управление сервером, переходы между текстовыми и голосовыми топиками. Раздел личных сообщений должен включать экран выбора диалога, поиск собеседника и страницу чата. Раздел серверов должен включать создание, управление, редактирование, текстовый топик и голосовой топик.

F-12 Обновление интерфейса без ручной перезагрузки. После создания, изменения или удаления серверов, топиков, диалогов и сообщений интерфейс должен обновлять соответствующие списки и текущий экран без ручной перезагрузки страницы. Это требование связано с пользовательской моделью приложений реального времени и отличается Chatterly от статических интерфейсов, где пользователь должен вручную обновлять состояние.

3.2 Нефункциональные требования

Нефункциональные требования определяют условия, при которых функциональные возможности должны работать приемлемо для пользователя и быть пригодными для дальнейшего развития проекта.

NF-1 Доступность через браузер. Приложение должно работать как веб-система и не требовать установки отдельного desktop-клиента. Это требование следует из ограничений TeamSpeak и из сильных сторон Jitsi Meet: пользователь должен открыть приложение в браузере, пройти аутентификацию и получить доступ к чатам и звонкам.

NF-2 Простота пользовательского сценария. Основные действия должны выполняться с небольшим числом шагов: регистрация, вход, поиск пользователя, создание DM, создание сервера, вступление в сервер, переход в топик, отправка сообщения и подключение к звонку. В MVP не вводится сложная система ролей, федерации и расширенных настроек, поскольку анализ Discord и Matrix / Element показал, что такая функциональность повышает порог входа.

NF-3 Сохранность и разделение данных. Пользователи, серверы, участники, топика, личные сообщения и сообщения топиков должны храниться в PostgreSQL. Пользовательские изображения должны храниться в S3-совместимом объектном хранилище. Redis должен использоваться для сессий, доставки событий и временного состояния звонков, но не как единственное хранилище постоянной истории общения.

- NF-4 **Производительность real-time взаимодействия.** MVP должен поддерживать не менее десяти активных пользователей в одном чате или звонке без заметной пользовательской задержки. Для текстового real-time взаимодействия используются WebSocket и Redis pub/sub, для истории сообщений – индексы и cursor-based пагинация, для медиа – WebRTC. Проверка данного требования должна выполняться в разделе тестирования через сценарии одновременной отправки сообщений и подключения нескольких участников к голосовому топику.
- NF-5 **Масштабируемость архитектуры.** Архитектура должна допускать дальнейшее увеличение числа пользователей, сообщений и WebSocket-соединений. Для этого REST API, WebSocket manager, сервисы бизнес-логики, PostgreSQL, Redis и S3-совместимое хранилище должны быть разделены как самостоятельные компоненты. Real-time события должны доставляться через Redis-каналы пользователя, а состояние голосового топики должно учитывать ownership backend-узла, что оставляет возможность запуска нескольких экземпляров серверной части.
- NF-6 **Безопасность и контроль доступа.** Пароли должны хэшироваться с использованием bcrypt, при проверке пароля должна учитываться соль на основе логина. Сессия должна храниться в Redis и передаваться через cookie с флагами HttpOnly и SameSite Lax. Все операции с DM, серверами, топиками, сообщениями и звонками должны проверять права пользователя на серверной стороне; клиентские проверки не должны считаться достаточными.
- NF-7 **Поддерживаемость кода.** Серверная часть должна быть разделена на доменные модели, сервисы бизнес-логики, адаптеры к PostgreSQL, Redis и S3, HTTP handlers, WebSocket handlers и composition root. Клиентская часть должна быть разделена по пользовательским областям: auth, dm, server, chat и voice. Бизнес-логика не должна концентрироваться в cmd/main.go; этот файл должен отвечать за конфигурацию и сборку зависимостей.
- NF-8 **Наблюдаемость и диагностируемость.** Приложение должно логировать HTTP-запросы и ошибки серверной части. Ошибки WebSocket и WebRTC signaling должны возвращаться клиенту в структурированном виде через событие `error` или понятный ответ REST API. Сбор метрик времени ответа, числа активных WebSocket-соединений и активных звонков относится к дальнейшему развитию, но архитектура не должна препятствовать его добавлению.

Сформулированные требования устраняют недостаток отчёта по практике, где связь между анализом аналогов и проектированием была недостаточно явной. Таблица ?? показывает происхождение требований, а сами требования задают проверяемые пользовательские сценарии и технические ограничения. Далее эти требования используются как основание для описания архитектуры, модели данных, API, real-time механизма и проверки реализации.

4 Архитектура и проектирование

4.1 Общая архитектура

Chatterly спроектирован как централизованное клиент-серверное веб-приложение. Пользователь работает с интерфейсом в браузере; клиентская часть загружается как SolidJS-приложение и обращается к серверной части по REST API и WebSocket. Серверная часть реализована на Go и содержит HTTP-обработчики, сервисы бизнес-логики, адаптеры к внешним системам и менеджер соединений реального времени. Постоянное состояние хранится в PostgreSQL, сессии и события pub/sub – в Redis, пользовательские изображения – в S3-совместимом объектном хранилище. Для аудио- и видеосвязи используется WebRTC; STUN-сервер применяется при ICE-согласовании сетевых адресов.

Обычный пользовательский запрос обрабатывается синхронно. Клиентская часть отправляет JSON-запрос на маршрут `/v1/...`; промежуточный обработчик определяет пользователя по сессионной cookie, API-слой разбирает параметры и передаёт доменную команду в сервис. Сервис проверяет права доступа и инварианты предметной области, при необходимости выполняет операцию в транзакции и обращается к PostgreSQL через адаптер. Результат возвращается клиенту в виде JSON-ответа, а ошибки приводятся к единому формату.

События реального времени вынесены в отдельный асинхронный контур. После сохранения сообщения сервис формирует событие `message` и публикует его в Redis-каналы пользователей-получателей. WebSocket manager поддерживает подписку на канал текущего пользователя, получает событие из Redis и отправляет его активным соединениям браузера. Такой подход отделяет запись постоянного состояния от доставки уведомлений и допускает запуск нескольких экземпляров backend при общей Redis-инфраструктуре.

Точка сборки приложения находится в `cmd/main.go`. В этом файле создаются конфигурация, logger, подключения к PostgreSQL, Redis и S3, менеджер транзакций, инфраструктурные адаптеры, in-memory stores, сервисы, фоновые синхронизаторы, WebSocket manager и HTTP server. Поэтому зависимости приложения инициализируются в одном месте, а осталь-

ные пакеты получают уже готовые интерфейсы.

Общая структура приложения представлена на рисунке ?? . Схема отделяет пользовательский браузер, серверную часть и внешние хранилища, а также показывает разные контуры взаимодействия: REST, WebSocket, Redis pub/sub и WebRTC.

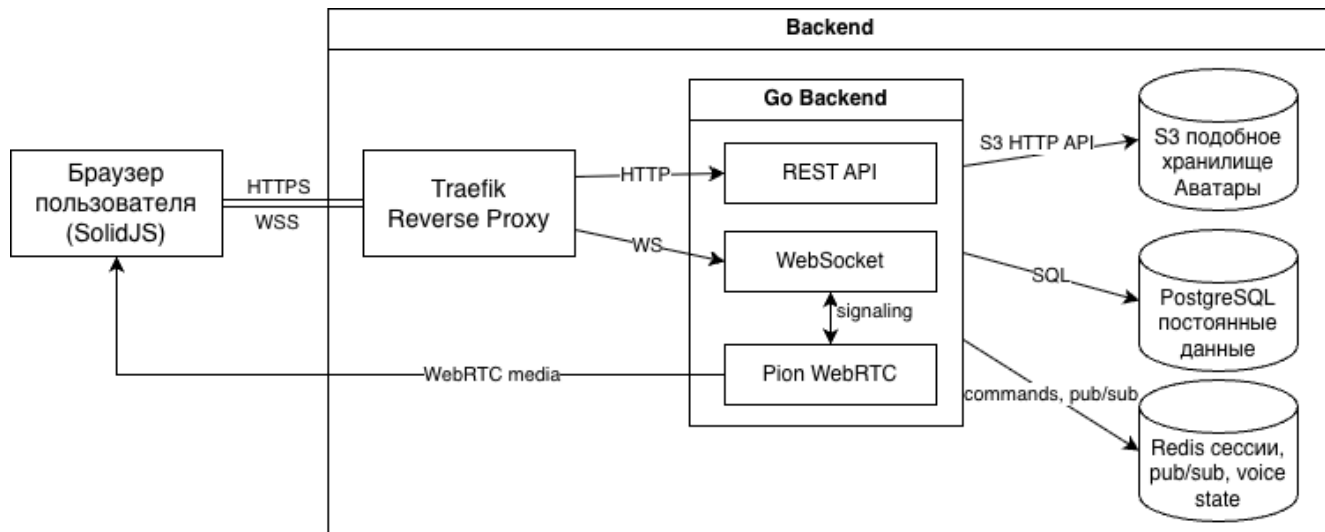


Рис. 4.1: Общая архитектура веб-приложения Chatterry

4.2 Модель данных

Модель данных разделена на три группы сущностей: пользователи, личные диалоги и серверы с топиками. Таблица **users** хранит идентификатор пользователя, публичное имя **username**, **login**, хэш пароля, идентификатор аватара и временные метки. Для **username** и **login** заданы уникальные ограничения, так как эти поля используются при поиске пользователя и аутентификации. Пароль хранится как результат bcrypt-хэширования, а исходное значение в базе данных отсутствует.

Личные сообщения представлены таблицами **dms**, **dm_participants** и **dm_messages**. Таблица **dms** фиксирует сам диалог и ссылку **last_message_id**, необходимую для построения списка диалогов без повторного вычисления последнего сообщения. Таблица **dm_participants** связывает пользователя с диалогом и хранит **last_read_message_id**, по которому определяется состояние прочтения. Сообщения хранятся в **dm_messages**; для загрузки истории используются индексы по **dm_id**, **created_at** и **id**, так как страницы выбираются в обратном хронологическом порядке.

Сообщества описываются четырьмя таблицами. **servers** хранит сами серверы, **server_participants** – состав участников, **topics** – текстовые и голосовые топики, **topic_messages** – историю текстовых топики. Участник сервера имеет роль **owner** или **member**: владелец

может изменять сервер и его топики, обычный участник получает доступ к существующим топикам. Активное состояние голосового топика является временным и хранится в памяти backend-узла и Redis, поэтому отдельной таблицы для медиасессий не требуется.

Целостность модели поддерживается уникальными ограничениями. Один пользователь не может быть добавлен в один и тот же ДМ или сервер более одного раза, а в рамках сервера не допускается два топика с одинаковым именем и типом. Связи, показанные на диаграмме, являются логическими: они используются в доменных сервисах и SQL-запросах, даже если часть ограничений контролируется не физическими внешними ключами, а прикладной логикой.

Логическая модель данных представлена на рисунке ??.

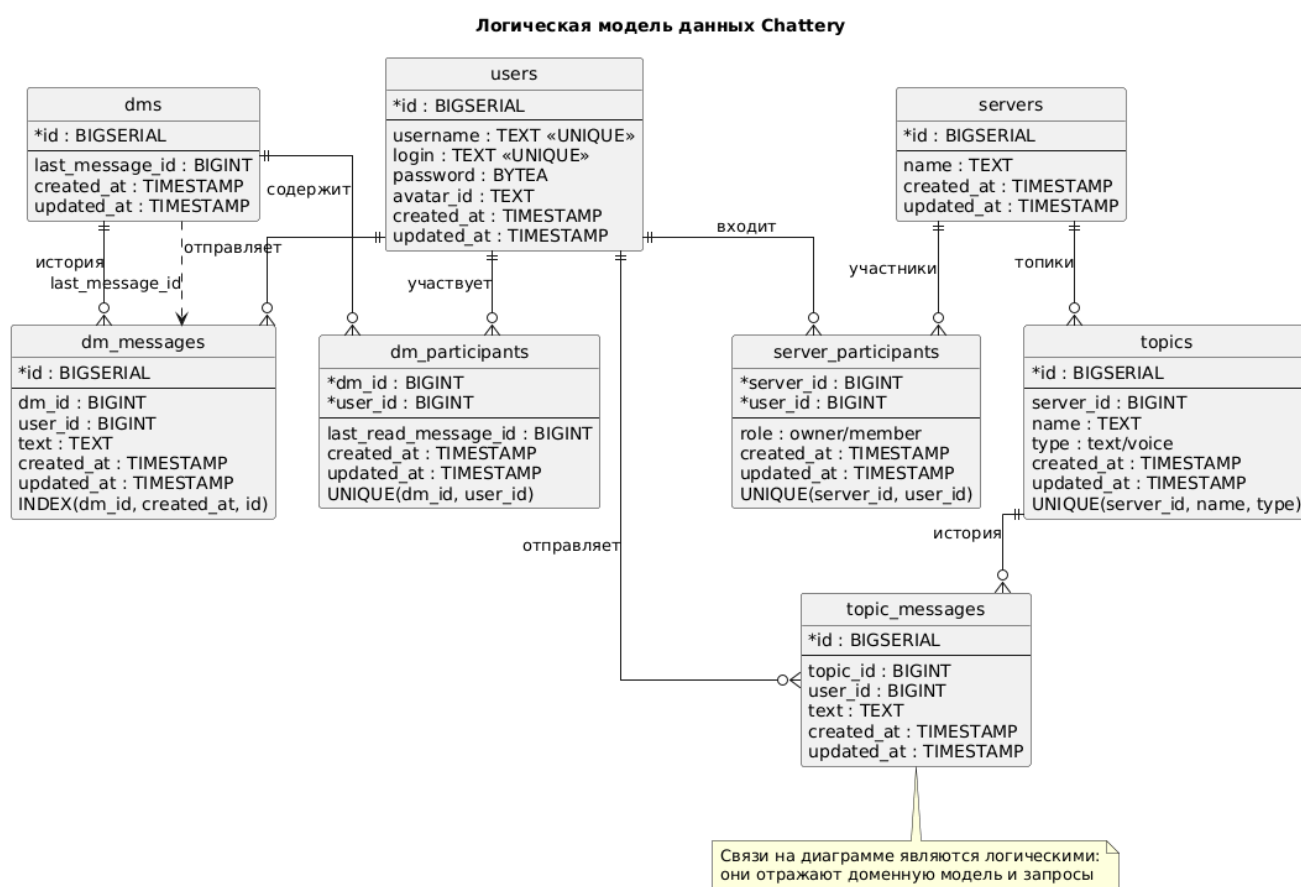


Рис. 4.2: Логическая модель данных Chatterry

4.3 Backend-архитектура

Серверная часть построена по слоистой схеме. Пакет `internal/domain` содержит основные сущности предметной области: `User`, `DM`, `DMMessage`, `Server`, `Topic`, `TopicMessage`, типизированные идентификаторы и объект-значение `Password`. Такой слой не зависит от HTTP, базы данных и Redis, поэтому он задаёт общий язык для API, сервисов и адаптеров.

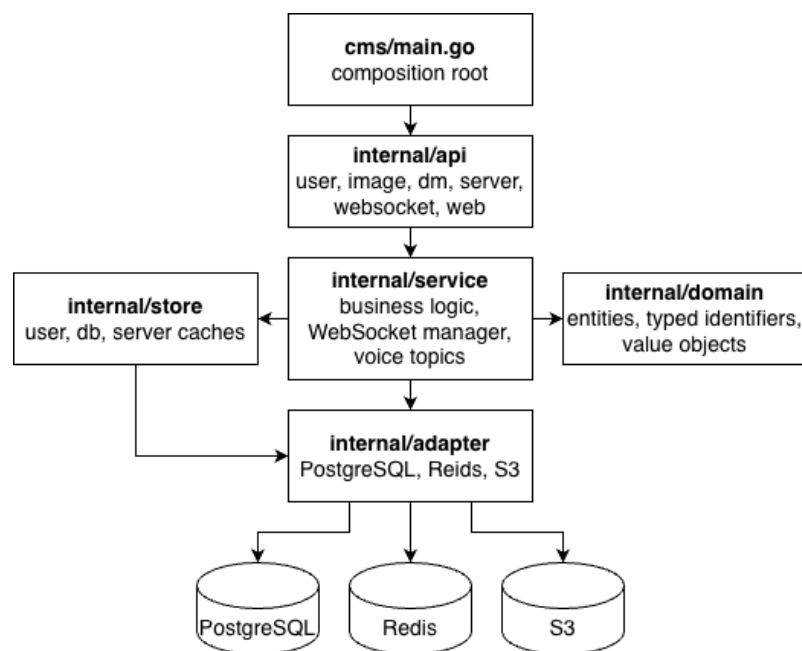


Рис. 4.3: Слоистая архитектура серверной части

Пакет `internal/api` реализует внешний интерфейс backend. В нём выделены обработчики пользователей, изображений, личных сообщений, серверов, WebSocket и отдачи статических страниц. HTTP-маршрутизация построена на `chi`; router используется совместно с промежуточными обработчиками для ограничения размера запроса, назначения request id, восстановления после panic, heartbeat и журналирования запросов. Доступ к защищённым маршрутам контролируется middleware пользовательского сервиса, который помещает идентификатор пользователя в request context.

Бизнес-логика размещена в `internal/service`. Отдельные сервисы отвечают за пользователей и сессии, изображения, DM, серверы, текстовые топики, WebSocket-соединения и голосовые топики. Сервисы обращаются к инфраструктуре через интерфейсы, поэтому детали PostgreSQL, Redis и S3 изолированы в пакете `internal/adaptor`. SQL-запросы описаны в `queries/*.sql` и компилируются в типизированный Go-код через `sqlc`; миграции базы данных находятся в `migrations`.

Для операций, состоящих из нескольких изменений, используется transaction manager. В транзакции выполняются создание DM с участниками, запись сообщения с обновлением последнего сообщения, создание сервера с владельцем, а также изменение структуры сервера и топики. Кроме того, в `internal/store` вынесены периодически синхронизируемые in-мемори представления пользователей, DM и серверов. Они применяются для быстрого поиска по идентификатору или имени и для формирования пользовательских данных в событиях.

Выбор Go обусловлен сетевым характером приложения: backend одновременно обслуживает HTTP-запросы, WebSocket-соединения, Redis pub/sub и WebRTC-сессии. Модель goroutine и стандартная библиотека HTTP позволяют реализовать эти задачи без отдельного сервера приложений. При этом слоистая структура ограничивает распространение конкурентной логики: она сосредоточена в фоновых синхронизаторах, WebSocket manager и сервисе голосовых топиков.

Разделение серверной части по слоям показано на рисунке ??.

4.4 WebSocket и доставка сообщений

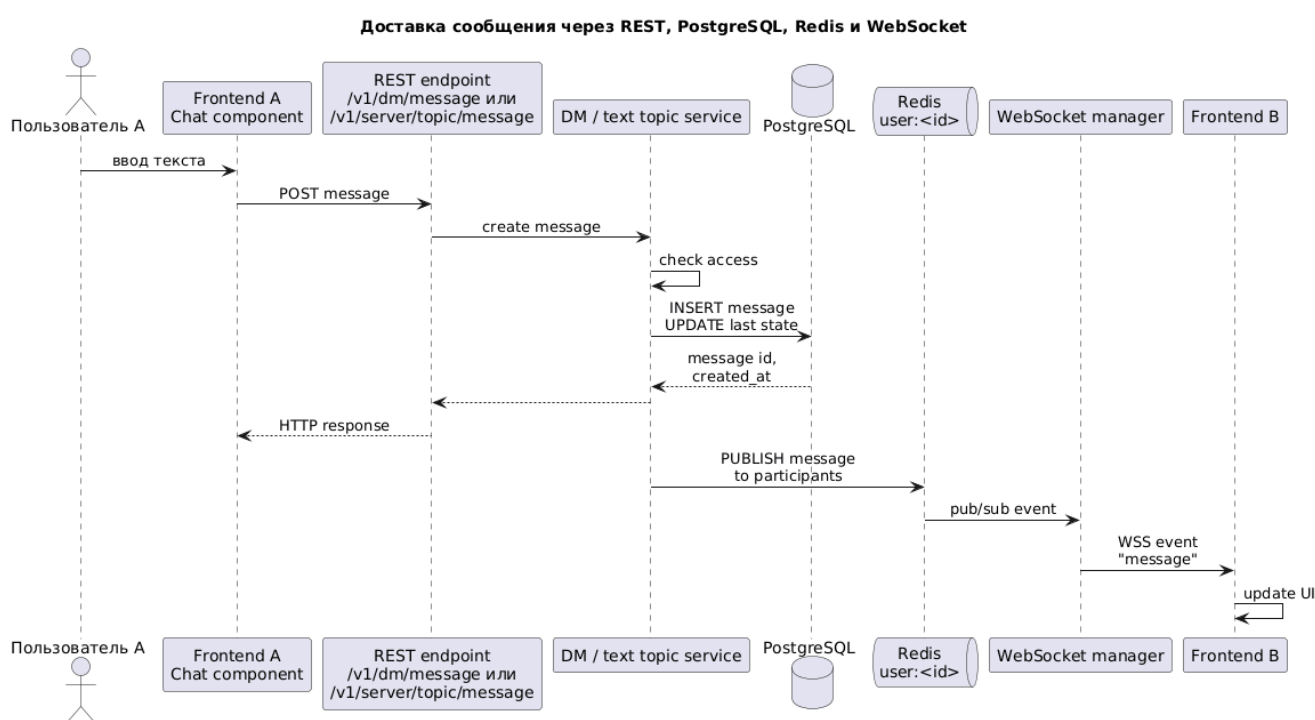


Рис. 4.4: Доставка сообщения через REST, PostgreSQL, Redis и WebSocket

WebSocket-маршрут `/ws/` доступен только аутентифицированному пользователю. После успешного upgrade backend создаёт объект соединения, регистрирует его в пользовательской сессии и запускает два независимых цикла: `ReadPump` для входящих событий и `WritePump` для исходящих событий. При закрытии соединения выполняется очистка состояния: соединение удаляется из сессии, WebSocket закрывается, а при нахождении пользователя в голосовом топике вызывается выход из комнаты.

Одна пользовательская сессия может содержать несколько WebSocket-соединений, например при открытии приложения в нескольких вкладках. Для такой сессии создаётся одна подписка Redis на канал `user:<id>`. Полученное событие рассылается активным соединениям пользователя. Для DM-событий доставка выполняется независимо от текущего открытого

канала, что позволяет показывать глобальные уведомления; для событий текстовых и голосовых топиков соединение должно быть подписано на соответствующий канал.

Подписка на канал выполняется клиентским событием `join`. Backend проверяет доступ к DM, текстовому топику или голосовому топику и только после этого сохраняет канал в состоянии соединения. Событие `leave` очищает текущую подписку. Для контроля жизнеспособности соединения сервер периодически отправляет `ping`; клиент отвечает `pong`. Если ответ не поступает в заданный интервал, соединение закрывается.

Отправка текстового сообщения остаётся REST-операцией. Клиент вызывает маршрут создания сообщения, сервис проверяет доступ, сохраняет сообщение в PostgreSQL и публикует событие `message` в Redis-каналы участников. WebSocket manager получает событие и доставляет его браузерам. Клиентская часть использует единое хранилище WebSocket-состояния: оно поддерживает переподключение, повторно отправляет `join` для активных каналов и временно буферизует сигнальные события, если соединение ещё не открыто.

Последовательность доставки текстового сообщения показана на рисунке ?? . Рисунок связывает REST-запрос отправителя, транзакционную запись сообщения и асинхронную доставку события получателям через Redis и WebSocket.

4.5 WebRTC и голосовые топики

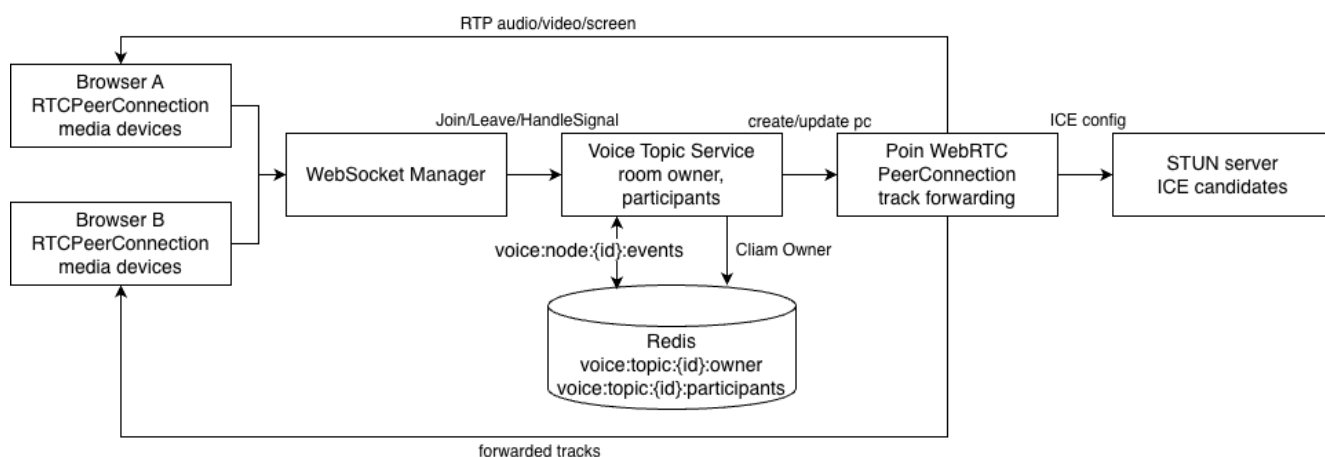


Рис. 4.5: Архитектура голосового топика и WebRTC signaling

Голосовой топик является временной комнатой звонка внутри сервера. Его постоянная часть хранится как запись `topics` с типом `voice`; активное состояние звонка не записывается в PostgreSQL. Им управляет voice topic service, который проверяет доступ пользователя к голосовому топику, ведёт список участников, обрабатывает сигнальные события и обслуживает WebRTC PeerConnection для каждого участника.

Для подготовки к горизонтальному масштабированию у каждой комнаты выбирается backend-узел-владелец. Идентификатор владельца хранится в Redis по ключу `voice:topic:<id>:owner` с TTL. Если событие `join`, `leave` или `signaling` приходит на другой backend-узел, оно публикуется во внутренний Redis-канал владельца `voice:node:<node>:events`. Активные участники дополнительно фиксируются в Redis sorted set `voice:topic:<id>:participants`; владелец периодически обновляет TTL владения комнатой и присутствие участников.

Сигнальная плоскость работает поверх WebSocket. Клиент и backend обмениваются событиями `voice_offer`, `voice_answer`, `voice_ice_candidate` и `voice_ice_candidates`. При входе пользователь получает `voice_state`, а остальные участники – события `voice_joined` и `voice_left`. Эти события не передают медиаданные; они нужны для согласования WebRTC-соединения и состояния UI.

Медиаплоскость реализована на backend с помощью Pion WebRTC [`pion_webrtc`]. Для каждого участника создаётся `PeerConnection`; в конфигурации регистрируются стандартные кодеки, STUN-сервер, при необходимости публичный IP и диапазон UDP-портов. Когда backend получает удалённый трек от одного участника, он создаёт локальный RTP-трек и добавляет его в `PeerConnection` других участников комнаты. При появлении или исчезновении треков выполняется renegotiation, а для восстановления видео запрашиваются ключевые кадры через RTCP.

Текущая схема не включает TURN-сервер. Это приемлемо для MVP, но при строгих NAT и корпоративных сетях WebRTC-соединение может не установиться или работать нестабильно. Добавление TURN-инфраструктуры относится к дальнейшему развитию системы.

Архитектура голосового топики показана на рисунке ???. Схема фиксирует две разные плоскости взаимодействия: сигнальные события проходят через WebSocket и Redis, а медиапотoki обрабатываются WebRTC-соединениями на стороне backend.

4.6 Frontend-архитектура

Клиентская часть реализована как SolidJS-приложение, собираемое Vite. Для страниц входа, регистрации и основного приложения используются отдельные точки входа, а основной router работает с base path `/app`. Маршруты DM включают `/app/dm`, `/app/dm/search` и `/app/dm/:dmId`. Для серверов предусмотрены маршруты списка, создания и управления, а также страницы текстового топики, голосового топики и редактирования сервера.

Код frontend разделён по функциональным областям. Модуль `auth` отвечает за про-

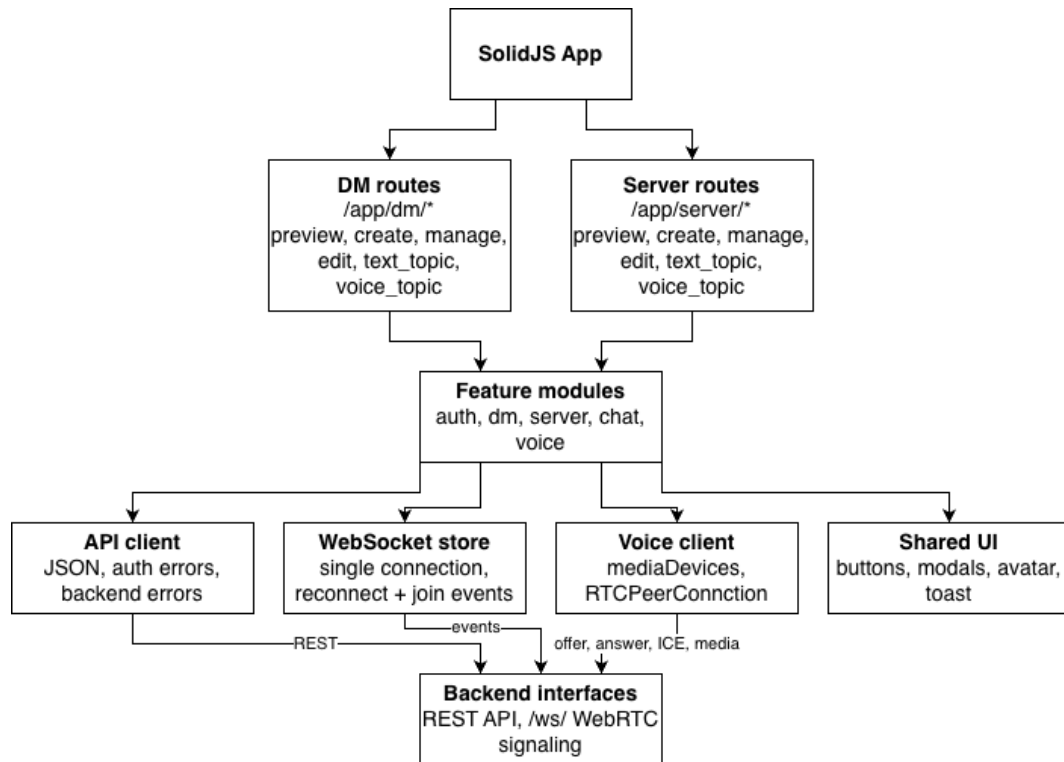


Рис. 4.6: Архитектура клиентской части и навигации

филь и аутентификационные действия, **dm** – за личные диалоги, **server** – за серверы и топики, **chat** – за общий интерфейс переписки, **voice** – за звонки. Общие элементы вынесены в **shared**: API-клиент, хранилище WebSocket-состояния, глобальное состояние пользователя, toast-уведомления, UI-компоненты и конфигурация маршрутов. Такое разделение уменьшает связность между пользовательскими сценариями и позволяет использовать один компонент чата как для DM, так и для текстовых топиков.

Общий API-клиент инкапсулирует вызов `fetch`, разбор JSON, сетевые ошибки, ошибку авторизации и ошибки backend. Хранилище WebSocket-состояния поддерживает одно соединение для приложения, набор подписчиков на события, обработчики сообщений и ошибок, карту активных каналов и очередь ожидающих сигнальных событий. При переподключении оно повторно отправляет `join` для активных каналов, поэтому маршруты не должны самостоятельно восстанавливать низкоуровневое соединение.

Компонент чата загружает первую страницу истории через REST API, догружает предыдущие сообщения при прокрутке вверх, дедуплицирует входящие сообщения и выполняет автопрокрутку вниз только при нахождении пользователя около конца списка. Отправка сообщения также выполняется через REST API, а обновление открытого чата и глобальные DM-уведомления приходят через WebSocket. Интерфейс голосового топика содержит локальную плитку, удалённых участников, статус звонка, меню микрофона, камеры и де-

монстрации экрана, а также настройки устройств ввода и вывода.

Структура клиентской части показана на рисунке ?? . Диаграмма связывает маршруты, feature-модули и общие клиентские сервисы, которые используются несколькими пользовательскими сценариями.

5 Реализация

5.1 Реализация backend

Исходный код сервиса размещён в репозитории GitHub [[chatterly_repo](#)]. Реализация разделена на backend-приложение, web-клиент, миграции базы данных, SQL-запросы и конфигурацию развёртывания.

Конфигурация backend реализована через переменные окружения. В неё входят параметры HTTP-сервера, подключения к PostgreSQL, Redis и S3-совместимому хранилищу, параметры сессий, ограничения изображений, лимит сообщений чата и настройки WebRTC. При запуске `cmd/main.go` считывает конфигурацию, создаёт внешние подключения и собирает сервисы приложения. HTTP-сервер использует middleware для журналирования запросов, восстановления после panic, ограничения размера запроса и проверки сессии пользователя.

Пользовательский сервис реализует регистрацию, вход, выход, получение текущего пользователя, обновление профиля и удаление аккаунта. Пароль сохраняется в виде bcrypt-хэша с добавлением login как соли. При успешном входе создаётся сессионный идентификатор, который записывается в Redis; в браузер передаётся HTTP-only cookie. При последующих запросах middleware читает cookie, проверяет сессию в Redis и помещает идентификатор пользователя в контекст запроса. Изменение login или пароля требует подтверждения текущим паролем.

Сервис изображений отвечает за аватары пользователей. Если пользователь не загрузил изображение, backend формирует identicon по имени пользователя. При загрузке файла проверяются размер и разрешение изображения, после чего оно приводится к JPEG и сохраняется в S3-совместимом хранилище. Удаление аватара очищает сохранённый идентификатор изображения и возвращает пользователя к автоматически сгенерированному изображению.

Сервис личных сообщений поддерживает поиск пользователей без существующего DM, создание диалога с двумя участниками, загрузку первой и следующих страниц истории, отметку сообщений как прочитанных и отправку сообщений. Создание сообщения выполня-

ется в транзакции: сообщение записывается в PostgreSQL, у диалога обновляется последнее сообщение, а для отправителя обновляется `last_read_message_id`. После фиксации состояния сервис публикует WebSocket-событие в Redis-каналы участников.

Сервис серверов реализует создание сервера, вступление и выход, обновление и удаление сервера, а также создание, изменение и удаление текстовых и голосовых топиков. Создатель сервера автоматически получает роль `owner`. Операции изменения структуры сервера доступны только владельцу; обычный участник может просматривать сервер и пользоваться его топиками. Текстовый topic service проверяет членство пользователя и тип топика, сохраняет сообщение в PostgreSQL и публикует событие всем участникам сервера.

WebSocket manager поддерживает несколько соединений одного пользователя, подписку пользовательской сессии на Redis-канал, обработку входящих событий и heartbeat. Цикл чтения принимает `join`, `leave`, `pong` и сигнальные события голосового топика; цикл записи отправляет события клиенту и периодически формирует `ping`. При закрытии соединения состояние очищается, а пользователь автоматически выходит из голосового топика, если находился в звонке.

Voice topic service реализует временные комнаты звонков для голосовых топиков. При входе пользователя проверяется доступ к топикам, выбирается backend-узел-владелец комнаты, участнику отправляется `voice_state`, а остальные получают `voice_joined` или `voice_left`. Сигнальная часть обрабатывает `offer`, `answer` и ICE candidates. Медиапоток обслуживается через Pion WebRTC [`pion_webrtc`]: backend принимает RTP-треки от участника, создаёт локальные треки для остальных участников, инициирует renegotiation при изменении состава треков и закрывает peer connection при выходе пользователя.

5.2 Реализация frontend

Клиентская часть содержит отдельные страницы входа и регистрации. Они используют отдельные точки входа Vite и вызывают API создания пользователя или входа. Основное приложение открывается под base path `/app`; его shell содержит router, глобальные провайдеры, состояние текущего пользователя, WebSocket-соединение и toast-уведомления. При входящем личном сообщении глобальный обработчик обновляет пользовательский интерфейс и показывает уведомление, если открыт другой чат.

DM-интерфейс включает sidebar со списком личных диалогов, страницу поиска пользователей и создание нового диалога из результата поиска. При получении события реального времени список диалогов обновляет preview последнего сообщения и состояние непрочитан-

ности. Для серверов реализованы список серверов пользователя, создание, поиск, присоединение, выход, редактирование и управление топиками. Навигация разделяет текстовые и голосовые топики.

Компонент чата используется как для DM, так и для текстовых топиков. При открытии чата он загружает первую страницу истории через REST API и подписывается на соответствующий WebSocket-канал. При прокрутке вверх загружаются более старые сообщения, входящие live-сообщения дедуплицируются, а автопрокрутка вниз выполняется только когда пользователь находится около конца списка. Отправка сообщения выполняется через REST API, после чего обновление состояния приходит через WebSocket.

Интерфейс голосового топика автоматически подключает пользователя к WebSocket-каналу и создаёт `RTCPeerConnection`. Пользователь может включать и выключать микрофон, камеру и демонстрацию экрана; локальные треки синхронизируются с peer connection. UI отображает локальную плитку, удалённых участников, статус звонка и ошибки доступа к устройствам. В настройках доступны камера, микрофон и устройство вывода звука; выбор speaker device используется только в браузерах, поддерживающих `setSinkId`.

Модальное окно профиля подключено к реальным endpoint-ам backend. Пользователь может обновить имя, login и пароль, загрузить новый аватар или удалить текущий. После успешного изменения frontend повторно запрашивает данные текущего пользователя и обновляет отображение аватара в sidebar, сообщениях и уведомлениях.

5.3 Инфраструктура и развёртывание

Для локальной разработки используется compose-конфигурация с PostgreSQL, Redis, MinIO, контейнером создания bucket и вспомогательным Redis UI. С помощью Makefile сделаны команды для поднятия локального окружения и применения миграций с помощью `Goose` [`goose`], live-reload для backend сервиса через `air` [`air`], и запуск Vite dev server для локальной разработки frontend, а так же выполняет генерация SQLc-кода для взаимодействия с PostgreSQL.

Непрерывная проверка реализована через GitHub Actions в workflow ci. Он запускается при pull request и при push в ветку master. Для всех job используется версия Go из `go.mod` и кэширование модулей. Job lint запускает `golangci-lint` версии v2.11.4 [`golangci_lint`], job test выполняет `go test ./...`, job build проверяет сборку командой `go build ./...`. Эти проверки фиксируют базовую компилируемость backend и отсутствие нарушений правил линтера перед обновлением сервиса.

Production-развёртывание выполнено через Dokploy [**dokploy**]. При обновлении кода сервис автоматически собирается и перезапускается на сервере. Backend-сборка использует multi-stage Dockerfile: на первом этапе Node/Vite собирает клиентские файлы, на втором этапе Go собирает бинарный файл `chattery`, после чего минимальный runtime-образ на Alpine запускает готовое приложение. Это позволяет поставлять frontend и backend как единый backend-артефакт.

Публичная точка входа приложения размещена на домене `chattery.space`. Внешний трафик принимает Traefik в сети Dokploy, TLS-сертификат выпускается через Let's Encrypt, а запросы направляются на HTTP-порт backend. Для повышения доступности и разделения WebRTC-портов развёрнуты два backend-экземпляра: `backend-1` и `backend-2`. Оба экземпляра имеют healthcheck `/health`; для WebRTC им выделены разные UDP-диапазоны, что согласуется с настройками `VOICE_ICE_UDP_PORT_MIN` и `VOICE_ICE_UDP_PORT_MAX`.

PostgreSQL и Redis не имеют публичного доступа и доступны backend только через закрытую внутреннюю сеть. Это снижает поверхность атаки: внешнему пользователю доступен только маршрутизатор приложения, а хранилища и pub/sub-инфраструктура остаются недоступными из публичного Интернета. S3-совместимое хранилище также подключается через отдельную private network; backend получает доступ к нему по внутреннему endpoint и использует публичные URL только для отображения пользовательских изображений.

6 Тестирование и оценка результата

6.1 Подход к тестированию

- написать, что для программного проекта требуется не только описать реализацию, но и показать корректность ключевых пользовательских сценариев;
- разделить проверки на несколько уровней: сборка, backend API, real-time события, frontend сценарии, звонки, безопасность доступа, производительность;
- честно указать состояние автотестов: в репозитории есть команда `go test ./...`, но unit/e2e тесты для сервисов и API handlers отмечены как незавершённые; если к финалу они будут добавлены, заменить этот пункт на описание конкретных тестов;
- привести таблицу сценариев тестирования, где для каждого сценария указать входные данные, ожидаемый результат и фактический статус.

6.2 Сценарии функционального тестирования

- **Регистрация и вход.** Создать пользователя, войти с правильным паролем, убедиться, что `/v1/user/me` возвращает текущий профиль; проверить ошибку при неверном пароле.
- **Сессии.** Проверить, что защищённые endpoints возвращают ошибку без cookie и работают после входа; проверить logout и невозможность повторного доступа по очищенной сессии.
- **Профиль и аватар.** Проверить получение identicon для пользователя без аватара, загрузку изображения допустимого размера, отказ при слишком большом файле или некорректном формате.
- **Личные сообщения.** Создать двух пользователей, найти второго пользователя, создать DM, отправить сообщение, проверить появление сообщения у обоих пользователей и обновление preview.
- **История DM.** Создать больше сообщений, чем `MESSAGES_LIMIT`, загрузить первую страницу и следующую страницу по cursor, проверить порядок и отсутствие дублей.
- **Серверы и топики.** Создать сервер, убедиться, что создатель получил роль owner, создать text topic и voice topic, изменить и удалить топик.
- **Права доступа.** Проверить, что пользователь, не состоящий в сервере, не может читать сообщения топика и подписываться на WebSocket-канал этого топика; проверить, что member не может изменять структуру сервера, если такое ограничение реализовано.
- **Текстовый топик.** Отправить сообщение в text topic и проверить его доставку всем участникам сервера через WebSocket.
- **WebSocket reconnect.** Разорвать соединение, дождаться переподключения frontend, проверить повторный join активного канала и получение новых сообщений.
- **Голосовой топик.** Открыть voice topic двумя пользователями, проверить voice_state, отображение участников, передачу микрофона, камеры и отключение участника при закрытии вкладки.
- **Демонстрация экрана.** Включить screen share, проверить появление видеопотока у другого участника и корректное прекращение трансляции.

6.3 Оценка производительности

- описать целевую метрику для чата: задержка от отправки сообщения одним пользователем до отображения у других участников;
- описать целевую метрику для WebSocket: число активных соединений, стабильность heartbeat, отсутствие массовых reconnect при десяти пользователях;
- описать целевую метрику для звонка: успешность подключения, время до появления удалённого участника, устойчивость аудио/видео при двух и более участниках;
- зафиксировать тестовую конфигурацию: машина, браузер, число пользователей, способ эмуляции пользователей, параметры PostgreSQL/Redis/backend;
- если нагрузочное тестирование не будет проведено, написать это как ограничение и оставить только ручную проверку сценариев MVP.

6.4 Оценка соответствия требованиям

- составить таблицу соответствия: требование, реализованный компонент, способ проверки, статус;
- для реализованных требований ссылаться на разделы реализации: user service, DM service, server service, text topic service, WebSocket manager, voice topic service, frontend routes;
- для частично реализованных требований указать причину и план завершения, например profile settings modal, автотесты, метрики, production deploy, TURN server;
- в выводе раздела написать, какие сценарии MVP подтверждены, а какие остаются перспективами развития.

7 Заключение

План итогового текста заключения:

- вернуться к цели из введения и написать, что в работе был спроектирован и реализован MVP веб-приложения Chatterly для группового общения;
- перечислить результаты:

- проведён анализ Discord, Zoom, Jitsi Meet, Matrix / Element и TeamSpeak;
 - сформированы функциональные и нефункциональные требования;
 - спроектирована архитектура backend, frontend, хранилищ, WebSocket и WebRTC;
 - реализованы пользователи, сессии, профили, аватары, серверы, топики, личные сообщения, история сообщений и доставка событий в реальном времени;
 - реализованы голосовые топики с WebRTC signaling, локальными media controls и отображением участников;
 - подготовлена локальная инфраструктура разработки на Docker Compose;
- описать практическую полезность: приложение демонстрирует независимую браузерную платформу для сообществ, объединяющую текстовое общение и звонки;
 - указать ограничения честно и конкретно: отсутствие production deploy, неполное автотестирование, необходимость TURN server для сложных сетей, необходимость расширить роли и модерацию, необходимость подключить или доработать все сценарии профиля, если они не будут завершены;
 - описать направления дальнейшего развития: e2e и unit tests, мониторинг и метрики, нагрузочное тестирование, мобильная адаптация, расширенные права доступа, push-уведомления, файлы в чатах, TURN-инфраструктура.